Presentation

Exercise_1_3323_.m

1. Δήλωση συνάρτησης main()

Το πρόγραμμα υλοποιείται μέσω της συνάρτησης main(), η οποία καλείται στο τέλος του αρχείου:

```
function main()
    ...
end
```

Η `main()` περιλαμβάνει όλα τα βασικά βήματα για την επίλυση των συστημάτων.

---

2. Ορισμός του πίνακα A και των διανυσμάτων b1, b2

Αμέσως μετά την έναρξη της συνάρτησης main(), ορίζεται ο τετραγωνικός πίνακας A:

```
A = [
    1  0 -1  0;
    0  2  3 -1;
   -1  3  2  2;
    0 -1  2  1
];
```

και τα διανύσματα–

```
b1 = [1; 0; -1; 1];
b2 = [0; -1; 0; 1];
```

Αυτά είναι τα συστήματα:

1. A x1 = b1

2. A x2 = b2

---

3. Παραγοντοποίηση LU (με πιθανή χρήση μεθόδου Gauss)

Η εντολή:

```
[L, U, P] = lu(A);
```

δημιουργεί τους πίνακες L, U και P, όπου:

- L είναι κάτω τριγωνικός (lower triangular) με 1 στην κύρια διαγώνιο.
- U είναι άνω τριγωνικός (upper triangular).
- P είναι πίνακας μεταθέσεων (permutation matrix), ο οποίος εφαρμόζεται για την αντιμετώπιση αριθμητικών δυσχερειών (π.χ. όταν απαιτείται να αλλάξουν μεταξύ τους κάποιες γραμμές κατά την απαλοιφή).

Η σχέση που ισχύει είναι:

P * A = L * U

---

4. Επίλυση κάθε συστήματος με εμπρόσθια και οπίσθια αντικατάσταση

Αφού έχουμε τους τριγωνικούς πίνακες L και U, μπορούμε να λύσουμε το σύστημα P*A x = Pb (μεταθέτουμε πρώτα το διάνυσμα b για λόγους συνέπειας) σε δύο φάσεις:

1. Λύνουμε το L y = P*b (εμπρόσθια αντικατάσταση).
2. Έπειτα λύνουμε το U x = y (οπίσθια αντικατάσταση).

Στον κώδικα, αυτό υλοποιείται ως εξής:

Για το b1

```
y1 = SolForward(L, P*b1);
x1 = SolBackward(U, y1);
```

Για το b2

```
y2 = SolForward(L, P*b2);
```

```
x2 = SolBackward(U, y2);
```

Ουσιαστικά, λόγω της σχέσης P*A* = *L*U, η επίλυση A x = b μετατρέπεται σε P*A* *x* = *P*b ⇒ L*U* *x* = *P*b. Γι' αυτό πρώτα λύνουμε L y = P*b και κατόπιν U x = y.

---

5. Εμφάνιση αποτελεσμάτων

Τέλος, εμφανίζονται τα διανύσματα λύσεων:

```
disp('Λύση για το b1:');
disp(x1);

disp('Λύση για το b2:');
disp(x2);
```

Συνάρτηση SolForward(L, b)

Η συνάρτηση SolForward πραγματοποιεί την εμπρόσθια αντικατάσταση (forward substitution) για να λύσει το σύστημα L y = b όταν ο L είναι κάτω τριγωνικός. Ο αλγόριθμος λειτουργεί γραμμικά από την πρώτη έως την τελευταία μεταβλητή:

1. Αρχικοποιούμε το διάνυσμα λύσης y με μηδενικά.
2. Για κάθε γραμμή i:
   - Θέτουμε y(i) = b(i).
   - Αφαιρούμε τα γινόμενα L(i,j)*y(j) για j από 1 έως i-1, ώστε να αφαιρεθεί η ήδη γνωστή συμβολή των προηγουμένων μεταβλητών.
   - Διαιρούμε τελικά το αποτέλεσμα με το διαγώνιο στοιχείο L(i,i).

Ο κώδικας φαίνεται παρακάτω:

```
function y = SolForward(L, b)
    n = length(b);        % Διάσταση του συστήματος
    y = zeros(n,1);       % Αρχικοποίηση διανύσματος y

    for i = 1:n
        y(i) = b(i);
        for j = 1:i-1
```

```
            y(i) = y(i) - L(i,j)*y(j);
        end
        y(i) = y(i) / L(i,i);
    end
end
```

Ροή υπολογισμού

- Στον πρώτο βρόχο (i=1), ουσιαστικά y(1) = b(1)/L(1,1).
- Στον επόμενο βρόχο (i=2), y(2) προσαρμόζεται αφαιρώντας το γινόμενο L(2,1)*y(1) και διαιρώντας με L(2,2).
- Με παρόμοιο τρόπο συνεχίζει μέχρι την τελευταία συνιστώσα y(n).

Συνάρτηση SolBackward(U, y)

Αντίστοιχα, η συνάρτηση SolBackward λύνει το U x = y, όπου ο U είναι άνω τριγωνικός. Εφαρμόζεται ο αλγόριθμος οπίσθιας αντικατάστασης (backward substitution):

1. Ξεκινάμε από την τελευταία εξίσωση (i = n) και προχωράμε προς τα πίσω.
2. Για κάθε γραμμή i (από κάτω προς τα πάνω):
    - Θέτουμε x(i) = y(i).
    - Αφαιρούμε τα γινόμενα U(i,j)*x(j) για j από i+1 έως n.
    - Διαιρούμε με το διαγώνιο στοιχείο U(i,i).

Ο κώδικας:

```
function x = SolBackward(U, y)
    n = length(y);         % Διάσταση του συστήματος
    x = zeros(n,1);        % Αρχικοποίηση διανύσματος x

    for i = n:-1:1
        x(i) = y(i);
        for j = i+1:n
            x(i) = x(i) - U(i,j)*x(j);
        end
        x(i) = x(i) / U(i,i);
```

```
    end
end
```

Ροή υπολογισμού

- Ξεκινάμε από i = n:
  x(n) = y(n)/U(n,n).
- Μετά πηγαίνουμε στο i = n-1:
  x(n-1) = (y(n-1) - U(n-1,n)*x(n)) / U(n-1,n-1).
- Συνεχίζουμε μέχρι το i=1.

Τι υπολογίζεται τελικά

Συνοψίζοντας:

1. Μέσω της `lu(A)`, βρίσκουμε την παραγοντοποίηση ώστε P*A* = *L*U.
2. Για κάθε b, το σύστημα A x = b γίνεται L U x = P b.
3. Με την εμπρόσθια αντικατάσταση βρίσκουμε y, όπου L y = P b.
4. Με την οπίσθια αντικατάσταση βρίσκουμε x, όπου U x = y.

Έτσι, παράγεται η τελική λύση x του συστήματος.

Πώς εκτελείται όλο το πρόγραμμα

1. Στο τέλος, καλείται η `main()`:

   ```
   main();
   ```

2. Γίνονται οι παραπάνω υπολογισμοί αυτόματα.
3. Εκτυπώνονται οι λύσεις x1 και x2 για τα αντίστοιχα διανύσματα b1 και b2.

---

output:

```
png@debian:~/Documents/Programming/Octave$octave Exercise_1_3
warning: function name 'main' does not agree with function fi
Λύση για το b1:
1.2500
```

```
-0.2500
0.2500
0.2500
Λύση για το b2:
0.071429
-0.357143
0.071429
0.500000
```

Exercise_2_3323_.m

Η Συνάρτηση `main()`

Η συνάρτηση `main()` είναι το σημείο εκκίνησης του προγράμματος. Εκτελεί τις ακόλουθες λειτουργίες:

1. **Ορισμός του Πίνακα `A`:**

   ```
   A = [1 0 -1 0;
        0 2 3 -1;
       -1 3 2 2;
        0 -1 2 1];
   ```

   Εδώ, ορίζεται ένας τετραγωνικός πίνακας `A` 4×4. Αυτός είναι ο πίνακας του οποίου θα βρούμε την κλιμακωτή μορφή και τον αντίστροφο.

2. **Μετατροπή σε Κλιμακωτή Μορφή:**

   ```
   [A_klimakoto, elementary_matrices] = toEchelon(A);
   fprintf('Κλιμακωτός Πίνακας:\\n');
   disp(A_klimakoto);
   ```

   Καλείται η συνάρτηση `toEchelon()` με είσοδο τον πίνακα `A`. Αυτή η συνάρτηση επιστρέφει δύο τιμές:

   - `A_klimakoto`: Ο πίνακας `A` μετασχηματισμένος σε κλιμακωτή μορφή.
   - `elementary_matrices`: Ένα cell array που περιέχει τις στοιχειώδεις μήτρες που χρησιμοποιήθηκαν για να μετατραπεί ο `A` σε κλιμακωτή μορφή.

Στη συνέχεια, εμφανίζεται ο κλιμακωτός πίνακας.

3. **Μετατροπή σε Ανοιγμένη Κλιμακωτή Μορφή:**

```
[A_anoigmenos_klimakotos, elementary_matrices] = toReduc
edEchelon(A_klimakoto, elementary_matrices);
fprintf('Ανοιγμένος Κλιμακωτός Πίνακας:\\n');
disp(A_anoigmenos_klimakotos);
```

Καλείται η συνάρτηση `toReducedEchelon()` με είσοδο τον κλιμακωτό πίνακα `A_klimakoto` και τις στοιχειώδεις μήτρες που υπολογίστηκαν προηγουμένως. Αυτή η συνάρτηση μετατρέπει τον κλιμακωτό πίνακα σε ανοιγμένη κλιμακωτή μορφή. Σημειώστε ότι οι στοιχειώδεις μήτρες ενημερώνονται με τις νέες μετασχηματίσεις. Εμφανίζεται ο ανοιγμένος κλιμακωτός πίνακας. Για έναν αντιστρέψιμο πίνακα, η ανοιγμένη κλιμακωτή μορφή είναι η ταυτοτική μήτρα.

4. **Υπολογισμός του Αντίστροφου:**

```
inverse_A = computeInverse(A, elementary_matrices);
fprintf('Ο αντίστροφος του A είναι:\\n');
disp(inverse_A);
```

Καλείται η συνάρτηση `computeInverse()` με τον αρχικό πίνακα `A` και τις συσσωρευμένες στοιχειώδεις μήτρες. Η λογική εδώ βασίζεται στο γεγονός ότι αν $E_1, E_2, ..., E_k$ είναι οι στοιχειώδεις μήτρες τέτοιες ώστε $E_k ... E_2 E_1 A = I$ (όπου I είναι η ταυτοτική μήτρα), τότε ο αντίστροφος του A είναι $A^{-1} = E_k ... E_2 E_1$.

5. **Έλεγχος του Αντίστροφου:**

```
identity_check = A * inverse_A;
fprintf('Έλεγχος A * A^(-1) = I:\\n');
disp(identity_check)
assert(norm(eye(size(A)) - identity_check, 'fro') < 1e-1
0)
```

Γίνεται ένας έλεγχος για να βεβαιωθεί ότι το γινόμενο του αρχικού πίνακα `A` με τον υπολογισμένο αντίστροφό του `inverse_A` είναι η ταυτοτική μήτρα. Η συνάρτηση `eye(size(A))` δημιουργεί την ταυτοτική μήτρα του ίδιου

μεγέθους με τον `A`. Η `norm` υπολογίζει την νόρμα Frobenius της διαφοράς, η οποία θα πρέπει να είναι πολύ μικρή (κοντά στο μηδέν) αν ο υπολογισμός είναι σωστός. Η `assert` ελέγχει αυτή τη συνθήκη και προκαλεί σφάλμα αν δεν ικανοποιείται.

Η Συνάρτηση `toEchelon(A)`

Η συνάρτηση `toEchelon(A)` μετατρέπει τον πίνακα `A` σε κλιμακωτή μορφή χρησιμοποιώντας στοιχειώδεις γραμμοπράξεις.

1. **Αρχικοποίηση:**

   ```
   A_klimakoto = A;
   n = size(A, 1);
   elementary_matrices = {};
   ```

   Δημιουργείται ένα αντίγραφο του πίνακα `A` με το όνομα `A_klimakoto`, το οποίο θα τροποποιείται. Το `n` αποθηκεύει τη διάσταση του πίνακα. Το `elementary_matrices` είναι ένα κενό cell array που θα αποθηκεύσει τις στοιχειώδεις μήτρες που αντιστοιχούν στις γραμμοπράξεις.

2. **Επανάληψη στις Στήλες:**

   ```
   for i = 1:n-1
       % ... κώδικας ...
   end
   ```

   Η εξωτερική επανάληψη τρέχει για κάθε στήλη του πίνακα, εκτός από την τελευταία. Σκοπός είναι να δημιουργηθούν μηδενικά κάτω από το κύριο στοιχείο (pivot) κάθε στήλης.

3. **Έλεγχος για Μικρό Διαγώνιο Στοιχείο:**

   ```
   if abs(A_klimakoto(i,i)) < 1e-10
       [A_klimakoto, E, found] = swapRows(A_klimakoto, i,
   n);
       if found
           elementary_matrices{end+1} = E;
       else
           continue
   ```

```
    end
end
```

Εάν το διαγώνιο στοιχείο `A_klimakoto(i,i)` είναι πολύ κοντά στο μηδέν (με ανοχή `1e-10`), καλείται η συνάρτηση `swapRows()` για να ανταλλάξει την τρέχουσα γραμμή με μια γραμμή από κάτω που έχει μη μηδενικό στοιχείο στην τρέχουσα στήλη. Η `swapRows()` επιστρέφει τον τροποποιημένο πίνακα, την αντίστοιχη στοιχειώδη μήτρα `E` και μια λογική μεταβλητή `found` που υποδεικνύει αν βρέθηκε γραμμή για ανταλλαγή. Αν βρεθεί, η στοιχειώδης μήτρα προστίθεται στο `elementary_matrices`. Αν δεν βρεθεί (δηλαδή όλα τα στοιχεία κάτω από το διαγώνιο είναι μηδέν), συνεχίζεται στην επόμενη στήλη.

4. **Μηδενισμός Στοιχείων Κάτω από το Pivot:**

```
for j = i+1:n
    factor = A_klimakoto(j,i) / A_klimakoto(i,i);
    A_klimakoto(j,:) = A_klimakoto(j,:) - factor * A_kli
makoto(i,:);
    E = eye(n);
    E(j, i) = -factor;
    elementary_matrices{end+1} = E;
end
```

Για κάθε γραμμή κάτω από την τρέχουσα γραμμή `i`, υπολογίζεται ένας συντελεστής `factor` έτσι ώστε όταν πολλαπλασιαστεί με την τρέχουσα γραμμή και αφαιρεθεί από την γραμμή `j`, το στοιχείο στη θέση `(j, i)` να γίνει μηδέν. Η αντίστοιχη στοιχειώδης μήτρα `E` δημιουργείται. Μια στοιχειώδης μήτρα για την αφαίρεση πολλαπλασίου μιας γραμμής από μια άλλη έχει όλα τα διαγώνια στοιχεία ίσα με 1, και ένα μη μηδενικό στοιχείο εκτός διαγωνίου στη θέση `(j, i)` ίσο με `-factor`.

Η Συνάρτηση `swapRows(A, i, n)`

Η συνάρτηση `swapRows(A, i, n)` ανταλλάσσει τη γραμμή `i` με μια γραμμή από κάτω που έχει μη μηδενικό στοιχείο στην στήλη `i`.

1. **Αναζήτηση Γραμμής για Ανταλλαγή:**

```
found = false;
for j = i+1:n
    if abs(A(j,i)) > 1e-10
        % ... κώδικας ανταλλαγής ...
        found = true;
        return
    end
end
```

Η συνάρτηση ψάχνει από την γραμμή `i+1` μέχρι την τελευταία γραμμή `n` για ένα στοιχείο στην στήλη `i` που είναι αρκετά μεγάλο (απόλυτη τιμή μεγαλύτερη από `1e-10`).

2. **Ανταλλαγή Γραμμών:**

```
temp = A(i,:);
A(i,:) = A(j,:);
A(j,:) = temp;
```

Αν βρεθεί κατάλληλη γραμμή `j`, οι γραμμές `i` και `j` ανταλλάσσονται.

3. **Δημιουργία Στοιχειώδους Μήτρας Ανταλλαγής:**

```
E = eye(n);
temp = E(i,:);
E(i,:) = E(j,:);
E(j,:) = temp;
```

Η στοιχειώδης μήτρα `E` που αντιστοιχεί στην ανταλλαγή των γραμμών `i` και `j` δημιουργείται ανταλλάσσοντας τις γραμμές `i` και `j` της ταυτοτικής μήτρας.

4. **Χειρισμός Περίπτωσης Μη Εύρεσης:**

```
E = [];
```

Αν δεν βρεθεί γραμμή για ανταλλαγή, η `found` παραμένει `false` και η `E` επιστρέφεται ως κενός πίνακας.

Η Συνάρτηση `toReducedEchelon(A, elementary_matrices)`

Η συνάρτηση `toReducedEchelon(A, elementary_matrices)` μετατρέπει τον κλιμακωτό πίνακα `A` σε ανοιγμένη κλιμακωτή μορφή.

1. **Μηδενισμός Στοιχείων Πάνω από τα Pivot:**

   ```
   for i = n:-1:2
       for j = i-1:-1:1
           factor = A(j, i) / A(i, i);
           A(j, :) = A(j, :) - factor * A(i, :);
           E = eye(n);
           E(j, i) = -factor;
           elementary_matrices{end+1} = E;
       end
   end
   ```

   Η εξωτερική επανάληψη τρέχει από την προτελευταία στήλη προς την πρώτη. Η εσωτερική επανάληψη τρέχει πάνω από το pivot της τρέχουσας γραμμής. Για κάθε στοιχείο πάνω από το pivot, υπολογίζεται ένας `factor` και γίνεται η κατάλληλη γραμμοπράξη για να μηδενιστεί το στοιχείο. Η αντίστοιχη στοιχειώδης μήτρα προστίθεται στο `elementary_matrices`.

2. **Κανονικοποίηση των Γραμμών (Κάνοντας τα Pivot Ίσα με 1):**

   ```
   for i = 1:n
       factor = 1 / A(i, i);
       A(i, :) = A(i, :) * factor;
       E = eye(n);
       E(i,i) = factor;
       elementary_matrices{end+1} = E;
   end
   ```

   Για κάθε γραμμή, το pivot (το πρώτο μη μηδενικό στοιχείο) διαιρείται με τον εαυτό του για να γίνει ίσο με 1. Αυτό επιτυγχάνεται πολλαπλασιάζοντας ολόκληρη τη γραμμή με τον αντίστροφο του pivot. Η αντίστοιχη στοιχειώδης μήτρα είναι μια διαγώνια μήτρα με όλα τα διαγώνια στοιχεία ίσα με 1 εκτός από το στοιχείο στη θέση `(i, i)` που είναι ίσο με `factor`.

Η Συνάρτηση `computeInverse(A, elementary_matrices)`

Η συνάρτηση `computeInverse(A, elementary_matrices)` υπολογίζει τον αντίστροφο του πίνακα `A` χρησιμοποιώντας τις στοιχειώδεις μήτρες.

1. **Αρχικοποίηση του Αντίστροφου:**

   ```
   n = size(A, 1);
   inverse_A = eye(n);
   ```

   Ο αντίστροφος αρχικοποιείται ως η ταυτοτική μήτρα.

2. **Πολλαπλασιασμός των Στοιχειωδών Μητρών (σε Αντίστροφη Σειρά):**

   ```
   for i = length(elementary_matrices):-1:1
       inverse_A = inverse_A * elementary_matrices{i};
   end
   ```

   Οι στοιχειώδεις μήτρες που αποθηκεύτηκαν κατά τη μετατροπή του `A` σε ανοιγμένη κλιμακωτή μορφή, αντιπροσωπεύουν τις στοιχειώδεις γραμμοπράξεις. Για να βρούμε τον αντίστροφο, πολλαπλασιάζουμε τις στοιχειώδεις μήτρες με αντίστροφη σειρά. Αυτό συμβαίνει επειδή αν $E_1A = B$, $E_2B = C$, τότε $E_2E_1A = C$. Για να λύσουμε για A, πολλαπλασιάζουμε με τους αντίστροφους των E_i με αντίστροφη σειρά: $A = E_1^{-1}E_2^{-1}C$. Στην περίπτωσή μας, η ακολουθία των E_i μετατρέπει τον A στην ταυτοτική μήτρα I, οπότε $E_k...E_2E_1A = I$, και $A^{-1} = E_k...E_2E_1$.

---

output:

```
png@debian:~/Documents/Programming/Octave$octave Exercise_2_3
warning: function name 'main' does not agree with function fi
Κλιμακωτός Πίνακας:
   1.0000         0   -1.0000         0
        0    2.0000    3.0000   -1.0000
        0         0   -3.5000    3.5000
        0         0         0    4.0000
Ανοιγμένος Κλιμακωτός Πίνακας:
   1   0   0   0
```

```
   0   1   0   0
   0   0   1   0
   0   0   0   1
Ο αντίστροφος του A είναι:
   0.964286   0.178571  -0.035714   0.250000
   0.178571   0.107143   0.178571  -0.250000
  -0.035714   0.178571  -0.035714   0.250000
   0.250000  -0.250000   0.250000   0.250000
Έλεγχος A * A^(-1) = I:
   1.0000        0        0        0
        0   1.0000        0        0
        0   0.0000   1.0000        0
   0.0000  -0.0000   0.0000   1.0000
```

Exercise_3_3323_.m

Συνάρτηση `main()`

```
function main()
    % Άσκηση 3: Ανάλυση Ιδιοτιμών και Ιδιοδιανυσμάτων
    % Συγγραφέας: Το Όνομά Σας
    % Ημερομηνία: Τρέχουσα Ημερομηνία

    % Δοκιμή της συνάρτησης με τον πίνακα B
    B = [1 5 10; 5 1 3; 10 3 1];
    disp('Ανάλυση για τον πίνακα B:');
    disp('Πίνακας B =');
    disp(B);
    [eigenvals_B, char_poly_B] = calculate_eigenvalues(B);
    disp('Συντελεστές χαρακτηριστικού πολυωνύμου:');
    disp(char_poly_B);
    disp('Ιδιοτιμές του B:');
    disp(eigenvals_B);

    % Ανάλυση για τον πίνακα A2
    A2 = [-1 5 2; 5 -1 2; 2 2 2];
```

```
    disp('Ανάλυση για τον πίνακα A2:');
    disp('Πίνακας A2 =');
    disp(A2);
    [eigenvals_A2, char_poly_A2] = calculate_eigenvalues(A
2);
    disp('Ιδιοτιμές του A2:');
    disp(eigenvals_A2);

    % Υπολογισμός ιδιοδιανυσμάτων για τον A2
    [V, D] = eig(A2);
    disp('Ιδιοδιανύσματα του A2:');
    disp(V);

    % Έλεγχος αν ο A2 είναι διαγωνοποιήσιμος
    if rank(V) == 3
        disp('Ο A2 είναι διαγωνοποιήσιμος επειδή έχει πλήρε
ς σύνολο γραμμικά ανεξάρτητων ιδιοδιανυσμάτων');
    else
        disp('Ο A2 δεν είναι διαγωνοποιήσιμος');
    end

    % Υπολογισμός αντιστρόφου του A2
    if det(A2) ~= 0
        A2_inv = inv(A2);
        disp('\\n\\n');
        disp('Αντίστροφος του A2:');
        disp(A2_inv);
    else
        disp('Ο πίνακας A2 είναι ιδιομορφικός και δεν μπορε
ί να αντιστραφεί.');
    end

    % Ανάλυση για τον πίνακα A από την Άσκηση 1
    A = [1 0 -1 0; 0 2 3 -1; -1 3 2 2; 0 -1 2 1];
    [V_A, D_A] = eig(A);
    disp('Ιδιοτιμές του A:');
    disp(diag(D_A));
    disp('Ιδιοδιανύσματα του A:');
```

```
    disp(V_A);

    % Επίλυση συστημάτων χρησιμοποιώντας ιδιοδιανύσματα
    b1 = [1; 0; -1; 1];
    b2 = [0; -1; 0; 1];

    sol_b1 = V_A * inv(D_A) * inv(V_A) * b1;
    sol_b2 = V_A * inv(D_A) * inv(V_A) * b2;

    disp('Λύση για το b1:');
    disp(sol_b1);
    disp('Λύση για το b2:');
    disp(sol_b2);
end
```

- `function main()`: Ορίζει τη κύρια συνάρτηση του προγράμματος. Όλος ο κώδικας που εκτελείται ξεκινάει από εδώ.
- `% Άσκηση 3: Ανάλυση Ιδιοτιμών και Ιδιοδιανυσμάτων`: Σχόλιο που περιγράφει τον σκοπό του κώδικα.
- `% Συγγραφέας: Το Όνομά Σας`: Σχόλιο που υποδεικνύει τον συγγραφέα του κώδικα.
- `% Ημερομηνία: Τρέχουσα Ημερομηνία`: Σχόλιο που υποδεικνύει την ημερομηνία δημιουργίας/τροποποίησης.
- `B = [1 5 10; 5 1 3; 10 3 1];`: Δημιουργεί έναν πίνακα 3x3 με όνομα `B` και τον αρχικοποιεί με τις συγκεκριμένες τιμές. Τα ερωτηματικά (`;`) χωρίζουν τις γραμμές του πίνακα.
- `disp('Ανάλυση για τον πίνακα B:');`: Εμφανίζει το κείμενο 'Ανάλυση για τον πίνακα B:' στην γραμμή εντολών. Η συνάρτηση `disp` χρησιμοποιείται για την εμφάνιση τιμών και κειμένου.
- `disp('Πίνακας B =');`: Εμφανίζει το κείμενο 'Πίνακας B ='.
- `disp(B);`: Εμφανίζει τα περιεχόμενα του πίνακα `B`.
- `[eigenvals_B, char_poly_B] = calculate_eigenvalues(B);`: Καλεί τη συνάρτηση `calculate_eigenvalues` με είσοδο τον πίνακα `B`. Η συνάρτηση αυτή επιστρέφει δύο τιμές: τις ιδιοτιμές (που αποθηκεύονται στην `eigenvals_B`)

και τους συντελεστές του χαρακτηριστικού πολυωνύμου (που αποθηκεύονται στην `char_poly_B`).

- `disp('Συντελεστές χαρακτηριστικού πολυωνύμου:');`: Εμφανίζει το κείμενο 'Συντελεστές χαρακτηριστικού πολυωνύμου:'.
- `disp(char_poly_B);`: Εμφανίζει τους συντελεστές του χαρακτηριστικού πολυωνύμου του πίνακα `B`.
- `disp('Ιδιοτιμές του B:');`: Εμφανίζει το κείμενο 'Ιδιοτιμές του B:'.
- `disp(eigenvals_B);`: Εμφανίζει τις ιδιοτιμές του πίνακα `B`.
- `A2 = [-1 5 2; 5 -1 2; 2 2 2];`: Δημιουργεί έναν πίνακα 3x3 με όνομα `A2` και τον αρχικοποιεί.
- `disp('Ανάλυση για τον πίνακα A2:');`: Εμφανίζει το κείμενο 'Ανάλυση για τον πίνακα A2:'.
- `disp('Πίνακας A2 =');`: Εμφανίζει το κείμενο 'Πίνακας A2 ='.
- `disp(A2);`: Εμφανίζει τα περιεχόμενα του πίνακα `A2`.
- `[eigenvals_A2, char_poly_A2] = calculate_eigenvalues(A2);`: Καλεί τη συνάρτηση `calculate_eigenvalues` με είσοδο τον πίνακα `A2` και αποθηκεύει τα αποτελέσματα στις μεταβλητές `eigenvals_A2` και `char_poly_A2`.
- `disp('Ιδιοτιμές του A2:');`: Εμφανίζει το κείμενο 'Ιδιοτιμές του A2:'.
- `disp(eigenvals_A2);`: Εμφανίζει τις ιδιοτιμές του πίνακα `A2`.
- `[V, D] = eig(A2);`: Η συνάρτηση `eig` υπολογίζει τις ιδιοτιμές και τα ιδιοδιανύσματα του πίνακα `A2`. Επιστρέφει δύο μεταβλητές: `V`, η οποία είναι ένας πίνακας όπου οι στήλες είναι τα ιδιοδιανύσματα του `A2`, και `D`, η οποία είναι ένας διαγώνιος πίνακας όπου τα διαγώνια στοιχεία είναι οι αντίστοιχες ιδιοτιμές.
- `disp('Ιδιοδιανύσματα του A2:');`: Εμφανίζει το κείμενο 'Ιδιοδιανύσματα του A2:'.
- `disp(V);`: Εμφανίζει τον πίνακα των ιδιοδιανυσμάτων του `A2`.
- `if rank(V) == 3`: Ελέγχει αν η τάξη (rank) του πίνακα `V` (των ιδιοδιανυσμάτων) είναι ίση με 3. Για έναν πίνακα 3x3, αυτό σημαίνει ότι τα ιδιοδιανύσματα είναι γραμμικά ανεξάρτητα.
- `disp('Ο A2 είναι διαγωνοποιήσιμος επειδή έχει πλήρες σύνολο γραμμικά ανεξάρτητων ιδιοδιανυσμάτων');`: Εμφανίζει αυτό το μήνυμα αν ο `A2` είναι

διαγωνοποιήσιμος. Ένας πίνακας είναι διαγωνοποιήσιμος αν έχει ένα πλήρες σύνολο γραμμικά ανεξάρτητων ιδιοδιανυσμάτων.

- `else`: Αν η συνθήκη `rank(V) == 3` είναι ψευδής, τότε εκτελείται αυτό το τμήμα του κώδικα.
- `disp('Ο A2 δεν είναι διαγωνοποιήσιμος');`: Εμφανίζει αυτό το μήνυμα αν ο `A2` δεν είναι διαγωνοποιήσιμος.
- `if det(A2) ~= 0`: Ελέγχει αν η ορίζουσα (determinant) του πίνακα `A2` είναι διαφορετική από το μηδέν. Αν η ορίζουσα είναι μη μηδενική, ο πίνακας είναι αντιστρέψιμος.
- `A2_inv = inv(A2);`: Υπολογίζει τον αντίστροφο του πίνακα `A2` χρησιμοποιώντας τη συνάρτηση `inv` και τον αποθηκεύει στη μεταβλητή `A2_inv`.
- `disp('\\n\\n');`: Εμφανίζει δύο κενές γραμμές για μορφοποίηση της εξόδου.
- `disp('Αντίστροφος του A2:');`: Εμφανίζει το κείμενο 'Αντίστροφος του A2:'.
- `disp(A2_inv);`: Εμφανίζει τον αντίστροφο του πίνακα `A2`.
- `else`: Αν η συνθήκη `det(A2) ~= 0` είναι ψευδής, τότε εκτελείται αυτό το τμήμα του κώδικα.
- `disp('Ο πίνακας A2 είναι ιδιομορφικός και δεν μπορεί να αντιστραφεί.');`: Εμφανίζει αυτό το μήνυμα αν ο πίνακας `A2` είναι ιδιόμορφος (δηλαδή, η ορίζουσά του είναι μηδέν) και δεν μπορεί να αντιστραφεί.
- `A = [1 0 -1 0; 0 2 3 -1; -1 3 2 2; 0 -1 2 1];`: Δημιουργεί έναν πίνακα 4x4 με όνομα `A` και τον αρχικοποιεί.
- `[V_A, D_A] = eig(A);`: Υπολογίζει τις ιδιοτιμές και τα ιδιοδιανύσματα του πίνακα `A`. Οι ιδιοτιμές αποθηκεύονται ως διαγώνια στοιχεία στον πίνακα `D_A`, και τα ιδιοδιανύσματα αποθηκεύονται ως στήλες στον πίνακα `V_A`.
- `disp('Ιδιοτιμές του A:');`: Εμφανίζει το κείμενο 'Ιδιοτιμές του A:'.
- `disp(diag(D_A));`: Εμφανίζει τις ιδιοτιμές του πίνακα `A`. Η συνάρτηση `diag` εξάγει τα διαγώνια στοιχεία από τον πίνακα `D_A`.
- `disp('Ιδιοδιανύσματα του A:');`: Εμφανίζει το κείμενο 'Ιδιοδιανύσματα του A:'.
- `disp(V_A);`: Εμφανίζει τον πίνακα των ιδιοδιανυσμάτων του `A`.

- `b1 = [1; 0; -1; 1];`: Δημιουργεί ένα διάνυσμα στήλης με όνομα `b1` και το αρχικοποιεί.
- `b2 = [0; -1; 0; 1];`: Δημιουργεί ένα διάνυσμα στήλης με όνομα `b2` και το αρχικοποιεί.
- `sol_b1 = V_A * inv(D_A) * inv(V_A) * b1;`: Επιλύει το γραμμικό σύστημα `Ax = b1` χρησιμοποιώντας την ιδιοσυνάρτηση ανάλυση. Η λύση `x` δίνεται από τον τύπο `V * D^(-1) * V^(-1) * b`, όπου `V` είναι ο πίνακας των ιδιοδιανυσμάτων και `D` είναι ο διαγώνιος πίνακας των ιδιοτιμών.
- `sol_b2 = V_A * inv(D_A) * inv(V_A) * b2;`: Επιλύει το γραμμικό σύστημα `Ax = b2` με τον ίδιο τρόπο.
- `disp('Λύση για το b1:');`: Εμφανίζει το κείμενο 'Λύση για το b1:'.
- `disp(sol_b1);`: Εμφανίζει τη λύση του συστήματος για το `b1`.
- `disp('Λύση για το b2:');`: Εμφανίζει το κείμενο 'Λύση για το b2:'.
- `disp(sol_b2);`: Εμφανίζει τη λύση του συστήματος για το `b2`.
- `end`: Τερματίζει τον ορισμό της συνάρτησης `main`.

Συνάρτηση `calculate_eigenvalues()`

```
function [eigenvals, char_poly] = calculate_eigenvalues(A)
    % Έλεγχος αν ο πίνακας είναι 3x3 και συμμετρικός
    [m, n] = size(A);
    if m ~= 3 || n ~= 3
        error('Ο πίνακας πρέπει να είναι 3x3');
    end
    if ~isequal(A, A')
        error('Ο πίνακας πρέπει να είναι συμμετρικός');
    end

    % Υπολογισμός συντελεστών χαρακτηριστικού πολυωνύμου
    a = -trace(A);
    b = det(A(1:2,1:2)) + det(A(2:3,2:3)) + det([A(1,1) A
(1,3); A(3,1) A(3,3)]);
    c = -det(A);
    char_poly = [1 a b c];
```

```
    % Υπολογισμός ιδιοτιμών χρησιμοποιώντας τη συνάρτηση ro
ots
    eigenvals = roots(char_poly);
end
```

- `function [eigenvals, char_poly] = calculate_eigenvalues(A)`: Ορίζει μια συνάρτηση με όνομα `calculate_eigenvalues` που δέχεται έναν πίνακα `A` ως είσοδο και επιστρέφει δύο τιμές: τις ιδιοτιμές (`eigenvals`) και τους συντελεστές του χαρακτηριστικού πολυωνύμου (`char_poly`).
- `% Έλεγχος αν ο πίνακας είναι 3x3 και συμμετρικός`: Σχόλιο που περιγράφει τον σκοπό του επόμενου μπλοκ κώδικα.
- `[m, n] = size(A);`: Η συνάρτηση `size` επιστρέφει τις διαστάσεις του πίνακα `A`. Η `m` θα περιέχει τον αριθμό των γραμμών και η `n` τον αριθμό των στηλών.
- `if m ~= 3 || n ~= 3`: Ελέγχει αν ο αριθμός των γραμμών (`m`) ή ο αριθμός των στηλών (`n`) δεν είναι ίσος με 3. Ο `||` συμβολίζει την λογική πράξη "OR".
- `error('Ο πίνακας πρέπει να είναι 3x3');`: Αν η συνθήκη είναι αληθής, η συνάρτηση `error` σταματά την εκτέλεση του προγράμματος και εμφανίζει το συγκεκριμένο μήνυμα σφάλματος.
- `if ~isequal(A, A')`: Ελέγχει αν ο πίνακας `A` δεν είναι ίσος με τον ανάστροφό του (`A'`). Η συνάρτηση `isequal` συγκρίνει δύο πίνακες και επιστρέφει λογική τιμή (true ή false). Το `~` συμβολίζει την λογική πράξη "NOT". Ένας συμμετρικός πίνακας είναι ίσος με τον ανάστροφό του.
- `error('Ο πίνακας πρέπει να είναι συμμετρικός');`: Αν ο πίνακας δεν είναι συμμετρικός, η συνάρτηση `error` σταματά την εκτέλεση του προγράμματος και εμφανίζει το μήνυμα σφάλματος.
- `% Υπολογισμός συντελεστών χαρακτηριστικού πολυωνύμου`: Σχόλιο που περιγράφει τον σκοπό του επόμενου μπλοκ κώδικα.
- `a = -trace(A);`: Υπολογίζει τον πρώτο συντελεστή του χαρακτηριστικού πολυωνύμου. Για έναν 3x3 συμμετρικό πίνακα, ο συντελεστής του `λ^2` είναι `trace(A)`, όπου `trace(A)` είναι το άθροισμα των διαγώνιων στοιχείων του `A`.
- `b = det(A(1:2,1:2)) + det(A(2:3,2:3)) + det([A(1,1) A(1,3); A(3,1) A(3,3)]);`: Υπολογίζει τον δεύτερο συντελεστή του χαρακτηριστικού πολυωνύμου.

Αυτός ο συντελεστής είναι το άθροισμα των κύριων ελασσόνων τάξης 2.

  - `det(A(1:2,1:2))`: Η ορίζουσα του υποπίνακα που αποτελείται από τις πρώτες δύο γραμμές και τις πρώτες δύο στήλες.
  - `det(A(2:3,2:3))`: Η ορίζουσα του υποπίνακα που αποτελείται από τις τελευταίες δύο γραμμές και τις τελευταίες δύο στήλες.
  - `det([A(1,1) A(1,3); A(3,1) A(3,3)])`: Η ορίζουσα του υποπίνακα που αποτελείται από τα στοιχεία στις θέσεις (1,1), (1,3), (3,1), και (3,3).

- `c = -det(A);`: Υπολογίζει τον σταθερό όρο του χαρακτηριστικού πολυωνύμου, ο οποίος είναι η αρνητική ορίζουσα του πίνακα `A`.
- `char_poly = [1 a b c];`: Δημιουργεί ένα διάνυσμα που περιέχει τους συντελεστές του χαρακτηριστικού πολυωνύμου. Για έναν 3x3 πίνακα, το χαρακτηριστικό πολυώνυμο έχει τη μορφή `λ^3 + aλ^2 + bλ + c`.
- `% Υπολογισμός ιδιοτιμών χρησιμοποιώντας τη συνάρτηση roots`: Σχόλιο που περιγράφει τον σκοπό του επόμενου μπλοκ κώδικα.
- `eigenvals = roots(char_poly);`: Η συνάρτηση `roots` υπολογίζει τις ρίζες ενός πολυωνύμου, του οποίου οι συντελεστές δίνονται στο διάνυσμα `char_poly`. Οι ρίζες αυτού του πολυωνύμου είναι οι ιδιοτιμές του πίνακα `A`.
- `end`: Τερματίζει τον ορισμό της συνάρτησης `calculate_eigenvalues`.

---

output:

```
png@debian:~/Documents/Programming/Octave$octave Exercise_3_3
warning: function name 'main' does not agree with function fi
Ανάλυση για τον πίνακα B:
Πίνακας B =
    1    5   10
    5    1    3
   10    3    1
Συντελεστές χαρακτηριστικού πολυωνύμου:
     1    -3  -131  -167
Ιδιοτιμές του B:
   13.5649
   -9.2312
   -1.3336
Ανάλυση για τον πίνακα A2:
```

```
Πίνακας A2 =
  -1   5   2
   5  -1   2
   2   2   2
Ιδιοτιμές του A2:
   6
  -6
   0
Ιδιοδιανύσματα του A2:
  -7.0711e-01  -4.0825e-01  -5.7735e-01
   7.0711e-01  -4.0825e-01  -5.7735e-01
   6.1059e-17   8.1650e-01  -5.7735e-01
Ο A2 είναι διαγωνοποιήσιμος επειδή έχει πλήρες σύνολο γραμμικ
Ο πίνακας A2 είναι ιδιομορφικός και δεν μπορεί να αντιστραφεί
Ιδιοτιμές του A:
  -2.4819
   0.9466
   2.2586
   5.2766
Ιδιοδιανύσματα του A:
  -0.182388   0.952979  -0.168981   0.173237
   0.541147   0.116380  -0.559860  -0.616581
  -0.635052   0.050849   0.212676  -0.740873
   0.520195   0.275138   0.782797  -0.202300
Λύση για το b1:
   1.2500
  -0.2500
   0.2500
   0.2500
Λύση για το b2:
   0.071429
  -0.357143
   0.071429
   0.500000
```